



# PyramidAttnCNN Architecture and Hyperparameter Optimization for IMU-Based Joint Biomechanics Prediction

Your Name

January 8, 2026

## 1 Neural Network Architecture: PyramidAttnCNN

### 1.1 Architecture Overview and Design Philosophy

The PyramidAttnCNN architecture is a multi-branch convolutional neural network specifically designed for time-series prediction of knee joint kinematics and kinetics from wearable IMU sensor data. The architecture addresses several key challenges in biomechanics prediction: capturing temporal patterns at multiple time scales simultaneously (since gait events occur over different durations), efficiently processing long time sequences (101 timepoints) while maintaining computational feasibility, learning hierarchical representations from low-level sensor signals to high-level biomechanical patterns, and identifying which temporal phases of the gait cycle are most important for prediction through attention mechanisms.

The name "PyramidAttnCNN" reflects two key architectural components: the multi-branch structure with different kernel sizes operating in parallel (though "multi-scale" is more accurate than "pyramid"), and the attention mechanism that weights the importance of different temporal features. The architecture consists of five major stages: parallel multi-branch convolutional feature extraction, aggressive temporal downsampling for computational efficiency, feature concatenation and dimensionality reduction, multi-head self-attention for temporal importance weighting, and axis-specific prediction heads for the three-dimensional output.

The input to the network is a tensor of shape (batch\_size, 42, 101) representing 42 IMU sensor channels over 101 normalized time points. The output is three separate predictions of shape (batch\_size, 101) for each anatomical axis (X, Y, Z), which are concatenated to form the final output of shape (batch\_size, 303). This architecture learns to map high-frequency sensor measurements to smooth biomechanical quantities while preserving the temporal structure of the gait cycle.

### 1.2 Multi-Branch Parallel Convolution for Multi-Scale Feature Learning

The first stage of the architecture employs multiple parallel convolutional branches, each using different kernel sizes to capture temporal patterns at different scales. This multi-branch design is inspired by the inception architecture philosophy: rather than committing to a single temporal resolution, the network simultaneously extracts features at multiple resolutions and lets the learning process determine which scales are most informative for the prediction task.

For the optimal configuration discovered through hyperparameter search, the network uses two branches with kernel sizes 3 and 7. The kernel size determines the temporal receptive field—how many consecutive time points each convolutional filter examines. A kernel size of 3 captures fine-grained, rapid changes in sensor signals such as impact transients during heel strike or rapid direction changes during swing phase. These small kernels are sensitive to instantaneous acceleration patterns and high-frequency components of movement. A kernel size of 7 captures broader temporal patterns spanning longer portions of the gait cycle, such as the overall trajectory of a limb segment through stance phase or the gradual weight transfer pattern. These larger kernels integrate information over longer time windows and are less sensitive to momentary fluctuations.

The mathematical operation in each branch consists of a series of 1D temporal convolutions. For an input signal with  $C_{\text{in}}$  input channels and  $T$  time points, a convolutional layer with kernel size  $k$  and  $C_{\text{out}}$  output channels applies:

$$y[c, t] = \sigma \left( \sum_{c'=0}^{C_{\text{in}}-1} \sum_{i=0}^{k-1} w[c, c', i] \cdot x[c', t + i - \lfloor k/2 \rfloor] + b[c] \right) \quad (1)$$

where  $w$  are the learned filter weights,  $b$  are bias terms, and  $\sigma$  is the activation function (ReLU). The padding ensures that the output has the same temporal length as the input (same-padding), which is critical for maintaining the 101-timepoint structure through the initial layers.

Each branch contains three sequential convolutional layers with progressively increasing channel dimensions: 24, 48, 96 in the optimal configuration. This progressive channel expansion is a form of hierarchical feature learning. The first convolutional layer with 24 output channels learns low-level temporal patterns directly from the raw sensor signals, such as the characteristic shape of an accelerometer signal during foot contact or the gyroscope pattern during knee flexion. With only 24 channels, this layer focuses on the most fundamental, commonly occurring patterns across all sensors and all phases of gait. The second convolutional layer with 48 channels receives the 24 low-level features and combines them to form mid-level representations. These might capture relationships between multiple sensors, such as coordination patterns between shank and thigh movements, or phase-specific combinations like rapid deceleration of the shank combined with ground reaction force increase indicating heel strike. The third convolutional layer with 96 channels creates high-level, abstract representations that integrate information across multiple time points and sensor channels. These features might represent complex biomechanical states like mid-stance with knee flexed and body weight transferring forward.

This hierarchical learning happens naturally through the composition of convolutional layers. Each layer builds upon the features learned by the previous layer, creating increasingly abstract and task-specific representations. The mathematical intuition is that early layers learn a dictionary of basic temporal patterns (edges, peaks, slopes in the time-series), while deeper layers learn combinations of these patterns that are predictive of the target biomechanical variables.

Between each convolutional layer, the architecture includes batch normalization, ReLU activation, and dropout for regularization. Batch normalization normalizes activations to have zero mean and unit variance within each mini-batch:

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}, \quad y = \gamma \hat{x} + \beta \quad (2)$$

where  $\gamma$  and  $\beta$  are learned scale and shift parameters. This stabilizes training by preventing activation distributions from shifting dramatically as the network learns, and it provides a regularization effect similar to dropout. However, standard batch normalization can be **problematic when batch size is 1** (during evaluation or final training batches). The architecture uses `SafeBatchNorm1d`, a custom wrapper that bypasses normalization for single-sample batches by returning the input unchanged, preventing division-by-zero errors and unstable statistics from single samples.

The ReLU (Rectified Linear Unit) activation function introduces non-linearity between convolutional layers, enabling the network to learn complex, non-linear relationships between sensor inputs and biomechanical outputs. Without these non-linearities, stacking multiple convolutional layers would be mathematically equivalent to a single linear transformation, severely limiting representational capacity.

Dropout randomly sets a fraction of activations to zero during training with probability  $p$ . For the optimal configuration, dropout rates are 0.026 for convolutional layers and 0.0024 for fully connected layers (extremely low, indicating the model found other regularization mechanisms more effective). Dropout prevents co-adaptation of neurons, forcing the network to learn robust features that do not rely on specific combinations of activations, thereby improving generalization to unseen test data.

### 1.3 Aggressive Temporal Downsampling for Computational Efficiency

**After the three hierarchical convolutional layers in each branch, the architecture applies two aggressive downsampling operations using** strided convolutions. This is a critical design decision that dramatically reduces computational cost while preserving the essential temporal information needed for biomechanics prediction.

At this point in the network, each branch has processed the input through three convolutions while maintaining the temporal dimension at 101 time points. The feature maps have shape (batch\_size, 96, 101) for each branch. Processing these long sequences through attention mechanisms would be computationally prohibitive because self-attention has quadratic complexity in sequence length. For sequence length 101, this means computing attention over 10,201 pairwise relationships between time points.

The downsampling strategy applies two strided convolutions in sequence. First, a convolution with kernel size 3, stride 4, and padding 1 reduces the temporal dimension from 101 to approximately 25 time points. The stride parameter controls how many positions the convolutional filter advances between applications. With stride 4, the filter moves 4 positions at a time, effectively subsampling the feature sequence by a factor of 4. The output length is computed as:

$$T_{\text{out}} = \left\lfloor \frac{T_{\text{in}} + 2 \cdot \text{padding} - \text{kernel\_size}}{\text{stride}} \right\rfloor + 1 = \left\lfloor \frac{101 + 2 - 3}{4} \right\rfloor + 1 = 25 \quad (3)$$

Second, a convolution with kernel size 3, stride 3, and padding 1 further reduces from 25 to

approximately 8 time points:

$$T_{\text{out}} = \left\lfloor \frac{25 + 2 - 3}{3} \right\rfloor + 1 = 8 \quad (4)$$

The cumulative downsampling ratio is approximately 12.6:1 (101 to 8), reducing the self-attention complexity from 10,201 operations to 64 operations, a 160-fold reduction in computational cost. This makes the attention mechanism tractable while still providing meaningful temporal context.

The critical question is whether this aggressive downsampling loses important information. The answer is nuanced: for biomechanics prediction, the fine temporal resolution of 101 points is less critical after hierarchical feature extraction has already identified important patterns. The early convolutional layers have extracted features representing specific gait events and patterns. The attention mechanism needs to weight the relative importance of these events (for example, heel strike features are more important than mid-swing features for predicting peak knee moment), not to resolve fine temporal details. Eight time points provide sufficient temporal structure to represent the major phases of gait: initial contact, loading response, mid-stance, terminal stance, pre-swing, initial swing, mid-swing, and terminal swing. The attention mechanism operates on these coarse phase representations rather than the original fine-grained temporal resolution.

An alternative approach would use pooling operations (max pooling or average pooling) for downsampling instead of strided convolutions. However, the architecture uses strided convolutions instead of pooling because strided convolutions learn optimal downsampling strategies through their trainable weights, whereas pooling applies fixed, hand-crafted aggregation rules. The convolutional filters can learn to emphasize certain features while suppressing others during downsampling, essentially learning a task-specific compression. Additionally, the strided convolutions include batch normalization and ReLU activation, providing non-linearity and normalization that pooling lacks.

#### 1.4 Feature Concatenation and Dimensionality Reduction

After both branches complete their hierarchical feature extraction and downsampling, their outputs are concatenated along the channel dimension. If two branches each produce features of shape (batch\_size, 96, 8), concatenation yields (batch\_size, 192, 8). For the general case with multiple branches and final channel dimension  $C$  per branch, concatenation produces features with doubled channel count.

This concatenation combines information from all temporal scales. The network now has access to both fine-grained patterns (from small kernel branch) and coarse-grained patterns (from large kernel branch) simultaneously, allowing subsequent layers to learn which scale is most relevant for each part of the prediction task. For example, predicting the rapid loading phase during heel strike might rely heavily on fine-scale features, while predicting the slow weight transfer during mid-stance might rely more on coarse-scale features.

However, concatenation also doubles the channel dimension (96 per branch times 2 branches equals 192 channels), increasing computational cost for subsequent operations. The architecture addresses this through a dimensionality reduction layer called self.reduce, which applies a 1 by 1 convolution to halve the channel dimension. A 1 by 1 convolution has kernel size 1, meaning it operates independently at each time point without mixing information across time. Its purpose is to learn

linear combinations of channels, effectively performing a learned projection from high-dimensional space (192 channels) to lower-dimensional space (96 channels). This is computationally much cheaper than maintaining 192 channels through the expensive attention mechanism.

Mathematically, the 1 by 1 convolution computes:

$$y[c, t] = \sigma \left( \sum_{c'=0}^{C_{\text{in}}-1} w[c, c'] \cdot x[c', t] + b[c] \right) \quad (5)$$

Notice that there is no summation over kernel positions because the kernel size is 1. This operation is equivalent to applying a fully connected layer independently at each time point, learning which combinations of the concatenated multi-scale features are most useful for the downstream prediction task. The reduction preserves temporal structure (still 8 time points) while compressing the channel information to the most predictive features.

## 1.5 Multi-Head Self-Attention Mechanism

The attention mechanism is the most sophisticated component of the architecture, enabling the network to dynamically weight the importance of different temporal phases of the gait cycle for predicting joint biomechanics. Understanding attention requires building up from fundamental concepts.

Before introducing attention, consider how convolutional and pooling operations aggregate information. Convolutions apply the same learned filters everywhere, treating all temporal positions equally. Pooling operations (like the mean pooling used later in this architecture) average across time uniformly. However, different phases of the gait cycle have vastly different importance for predicting peak knee moment—the loading response phase where body weight rapidly transfers to the leg is far more important than the mid-swing phase where the leg moves freely through the air with minimal ground contact. Attention allows the network to learn these importance weightings automatically from data.

Self-attention computes a weighted average of all positions in a sequence, where the weights are dynamically determined based on the content at each position. Unlike fixed pooling weights, attention weights adapt to the specific input sample, allowing the network to focus on relevant features while suppressing irrelevant ones.

The attention mechanism transforms the reduced features from shape (batch\_size, 96, 8) to shape (batch\_size, 8, 96) using a permute operation, changing from (batch, channels, time) format to (batch, time, channels) format required by the attention module. This reinterpretation treats the 8 downsampled time points as a sequence of 96-dimensional feature vectors, where attention will learn relationships between different temporal positions.

Self-attention operates through three learned linear projections of the input: queries (Q), keys (K), and values (V). For an input sequence with  $T$  time points and  $d$  feature dimensions, these are computed as linear transformations where learned weight matrices project the input into query, key, and value spaces. The intuition behind these three projections is that queries represent what information each position is looking for, keys represent what information each position contains, and values contain the actual feature information to be aggregated.

The attention mechanism computes similarity scores between each query and all keys using scaled dot product:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (6)$$

Breaking this down step by step: the matrix multiplication  $QK^T$  computes pairwise dot products between all queries and keys, creating a matrix where entry  $(i, j)$  represents how much position  $i$  should attend to position  $j$ . High dot product indicates high similarity—position  $i$ ’s query aligns well with position  $j$ ’s key, suggesting that position  $j$  contains relevant information for position  $i$ . Division by  $\sqrt{d_k}$  scales the dot products to prevent them from growing too large when feature dimension  $d_k$  is large. Without scaling, softmax can saturate into near-one-hot distributions, reducing gradient flow during training. Softmax normalizes each row to a probability distribution, ensuring attention weights for each position sum to 1, creating a proper weighting scheme. Multiplication by  $V$  computes the weighted average: output at position  $i$  is the sum over all positions  $j$  of attention weight times value vector at  $j$ , aggregating value vectors according to attention weights.

For the gait biomechanics task, attention learns patterns like: when predicting knee moment at heel strike (position 1), strongly attend to the heel strike features (position 1 itself) and the subsequent loading response (position 2), while ignoring swing phase (positions 6 through 8). The learned attention weights encode these phase-specific dependencies automatically.

Rather than computing attention once, multi-head attention computes it multiple times in parallel with different learned projections, then concatenates the results. The optimal configuration uses 4 attention heads. Multiple heads allow the network to attend to different aspects of the temporal sequence simultaneously. One head might focus on rapid transitions (heel strike, toe-off), another head might track gradual weight transfer patterns, a third head might encode symmetry relationships between early and late stance, and the fourth head might capture swing phase dynamics. Each head learns a different attention pattern specialized for different aspects of the prediction task, and the final concatenation and linear projection combine these diverse perspectives into a unified representation.

After attention, a dropout layer (probability 0.1) randomly drops attention connections during training, preventing the network from becoming overly reliant on specific temporal dependencies and encouraging robust learning of multiple attention patterns.

## 1.6 Global Temporal Aggregation and Axis-Specific Prediction Heads

After the attention mechanism enriches the temporal features with learned importance weights, the sequence still has shape (batch\_size, 8, 96)—eight time points with 96-dimensional features at each point. To generate the final predictions, the architecture must somehow aggregate across these eight time points to create a fixed-dimensional representation that can be fed to prediction layers.

The aggregation strategy uses global average pooling across the temporal dimension, computing the mean of the 96-dimensional feature vectors across all 8 time points, resulting in a single 96-dimensional vector for each sample in the batch. This pooled representation summarizes the entire gait cycle by averaging the attention-weighted features across all temporal phases. Because the attention mechanism has already learned which phases are important, the pooling operation effec-

tively computes a learned weighted average where important phases (that received high attention weights) contribute more strongly to the pooled representation.

Mean pooling is preferred over max pooling or other aggregations because it preserves magnitude information about features rather than just their presence. For continuous biomechanical quantities like joint angles and moments, magnitude matters—predicting a knee flexion angle of 15 degrees versus 60 degrees represents fundamentally different movement patterns, not just the presence or absence of flexion.

The pooled representation feeds into three separate linear prediction heads, one for each anatomical axis (X, Y, Z). This multi-head design reflects the structure of the prediction problem: joint angles and moments are inherently three-dimensional quantities in anatomical coordinate systems, where the X, Y, and Z axes represent different planes of motion (sagittal, frontal, transverse). Each axis has different characteristics and potentially different optimal prediction strategies. For example, the sagittal plane (X-axis) dominates during walking with large flexion-extension movements, while frontal plane motion (Y-axis) is smaller but critical for balance control.

The three linear layers each learn an independent mapping from the 96-dimensional pooled features to 101-dimensional output representing predictions at all 101 time points of the normalized gait cycle. Having separate heads for each axis allows the network to learn axis-specific patterns. The weight matrix for X-axis learns which combinations of pooled features are predictive of sagittal plane motion, while Y-axis weight matrix might emphasize different feature combinations relevant for frontal plane motion. This specialization improves prediction quality compared to a single shared prediction head.

The three predictions are concatenated to form the final output of 303 dimensions containing the complete predicted time series: elements 0 through 100 are X-axis predictions, 101 through 201 are Y-axis, and 202 through 302 are Z-axis. During evaluation, this can be reshaped to (3, 101) for convenient visualization and comparison with ground truth.

## 1.7 Complete Forward Pass Summary

To solidify understanding, let us trace a single sample through the entire architecture. The input has shape (1, 42, 101) representing 42 IMU channels over 101 time points.

Branch 1 with kernel size 3 processes this through: Conv1D from 42 to 24 channels with kernel 3 producing shape (1, 24, 101) followed by BatchNorm, ReLU, and Dropout. Then Conv1D from 24 to 48 channels with kernel 3 producing shape (1, 48, 101) followed by BatchNorm, ReLU, and Dropout. Then Conv1D from 48 to 96 channels with kernel 3 producing shape (1, 96, 101) followed by BatchNorm, ReLU, and Dropout. Then Conv1D from 96 to 96 channels with kernel 3 and stride 4 producing shape (1, 96, 25) followed by BatchNorm and ReLU. Finally Conv1D from 96 to 96 channels with kernel 3 and stride 3 producing shape (1, 96, 8) followed by BatchNorm and ReLU.

Branch 2 follows the same structure but with kernel size 7, also producing shape (1, 96, 8).

Concatenation combines both branches to produce shape (1, 192, 8). Dimensionality reduction applies Conv1D from 192 to 96 channels with kernel 1 producing shape (1, 96, 8) followed by BatchNorm and ReLU.



Attention processing begins with permuting to shape (1, 8, 96). Multi-head attention with 4 heads maintains shape (1, 8, 96). Dropout is applied.

Global aggregation uses mean pooling over the time dimension to produce shape (1, 96). Three prediction heads each apply linear transformation from 96 to 101 dimensions. The X-axis head produces (1, 101), Y-axis head produces (1, 101), and Z-axis head produces (1, 101). These are concatenated to form the final output of shape (1, 303).

This complete architecture contains approximately 380,000 trainable parameters, which is moderate-sized for deep learning standards, making it trainable on the available dataset of 903 gait cycles with appropriate regularization.

## 2 Hyperparameter Optimization with Optuna

### 2.1 The Hyperparameter Search Problem

Neural network performance critically depends on numerous hyperparameter choices: architectural decisions (number of layers, kernel sizes, channel dimensions), regularization parameters (dropout rates, weight decay), optimization settings (learning rate, batch size, optimizer algorithm), and augmentation strategies (noise levels). The space of possible hyperparameter configurations is enormous—even with just 10 hyperparameters each with 5 possible values, there are nearly 10 million configurations. Manually searching this space is impractical, and naive random search is inefficient because most of the search space contains poor configurations.

The project employs Optuna, an automatic hyperparameter optimization framework that uses sophisticated search algorithms to efficiently explore the hyperparameter space. Optuna constructs a probabilistic model of the relationship between hyperparameters and model performance, then uses this model to intelligently suggest promising configurations to evaluate next, dramatically reducing the number of trials needed to find good hyperparameters compared to random or grid search.

### 2.2 Bayesian Optimization and Tree-Structured Parzen Estimator

Optuna’s default search algorithm is the Tree-Structured Parzen Estimator (TPE), a form of Bayesian optimization. The fundamental idea is to build a probabilistic model of performance given hyperparameters and use this model to guide the search toward regions of hyperparameter space likely to yield good performance.

Given a set of previous trials where hyperparameter configurations have been evaluated and performance measured, Bayesian optimization works by building a probabilistic model predicting performance for untested configurations, using the model to compute an acquisition function that balances exploration (trying uncertain regions) and exploitation (trying regions near known good configurations), selecting the next configuration to evaluate by maximizing the acquisition function, evaluating this configuration, adding the result to the trial history, and repeating.

TPE takes a unique approach to modeling the performance distribution. Instead of directly modeling the probability of performance given hyperparameters as in Gaussian Process-based Bayesian optimization, TPE models the probability of hyperparameters given performance level. Specifically, it creates two density models: one for the density of good configurations (hyperparameters

that produced performance better than some threshold), and one for the density of bad configurations (hyperparameters that produced worse performance). These densities are modeled using tree-structured Parzen windows, which handle mixed continuous and categorical hyperparameters naturally.

The acquisition function in TPE is the ratio of good density to bad density. Configurations with high ratio values are more likely to be in the good distribution than the bad distribution, making them promising candidates for evaluation. This approach is computationally efficient (evaluating the ratio is fast) and naturally handles different variable types (continuous learning rates, categorical optimizer choices, integer layer counts).

## 2.3 Optuna Implementation and Search Configuration

The hyperparameter search is implemented with Optuna study creation specifying direction to minimize validation loss, storage in a SQLite database for persistent trial history allowing interrupted searches to resume, and TPESampler with fixed seed for reproducible search sequences. The study optimization runs for a specified number of trials with garbage collection after each trial to manage memory.

The direction minimize specifies that the goal is to minimize the objective function value (validation loss). The storage parameter creates a SQLite database to persist all trial results, allowing the search to be interrupted and resumed without losing progress. The load if exists flag enables continuing previous searches, which is essential for long-running hyperparameter optimization that may span multiple days. The TPESampler with a fixed seed ensures reproducibility—running the search again with the same seed produces the same sequence of hyperparameter suggestions.

## 2.4 Hyperparameter Search Space Definition

The search space defines which hyperparameters to optimize and their possible values. Architectural hyperparameters include channel progression with options [24, 48, 96] for approximately 350,000 parameters or [32, 64, 128] for approximately 480,000 parameters. Both maintain the 2 times expansion pattern where each layer doubles the previous, which is a common design principle in CNNs. The search is constrained to these two options rather than exploring all possible channel combinations because preliminary experiments found that models exceeding 500,000 parameters tended to overfit the relatively small dataset of 903 training samples.

Kernel size combinations determine the temporal receptive fields of the parallel branches. The options are [3, 5] and [3, 7], both including a small kernel for fine-grained patterns plus either a medium or larger kernel for broader patterns. Kernels were not explored beyond size 7 because the aggressive downsampling means that even kernel size 7 only sees a limited temporal window after the first few layers.

The number of attention heads controls the diversity of attention patterns the model can learn. The search considers 2 or 4 heads. Fewer heads reduce computation and parameters but may limit the model’s ability to capture diverse temporal dependencies. More heads allow more specialized attention patterns but increase computation and risk overfitting.

Regularization hyperparameters include dropout rates for convolutional and fully connected layers. Rather than searching the entire range from 0 to 1, the search is focused around a baseline value

from previous successful experiments, exploring 0.5 times to 2 times the baseline. This focused search is more efficient than wide exploration when good approximate values are already known.

Weight decay (L2 regularization) penalizes large weight values during optimization. The search explores weight decay on a logarithmic scale covering multiple orders of magnitude. Logarithmic sampling is appropriate because weight decay values are typically very small and their effects are roughly proportional to their logarithm.

Optimization hyperparameters include learning rate which determines the step size during gradient descent optimization. It is arguably the most critical hyperparameter—too large causes training instability and divergence, too small causes slow convergence and getting stuck in poor local minima. The search explores learning rates on a logarithmic scale around a proven baseline value.

Batch size controls how many training samples are processed together in each gradient update. The search considers values 12, 16, and 20, which are constrained by GPU memory. Batch size affects training in multiple ways: larger batches provide more accurate gradient estimates but require fewer update steps per epoch, larger batches enable better GPU parallelization but may generalize worse, and batch size interacts with batch normalization where larger batches provide more stable statistics.

Some hyperparameters are held constant based on prior knowledge or to reduce search complexity. Three convolutional layers per branch provides sufficient hierarchical learning depth without excessive parameters. AdamW optimizer is chosen over standard Adam, SGD, or Nadam based on superior performance in preliminary tests. Huber loss combines the best properties of MSE and MAE, being quadratic for small errors and linear for large errors. Gradient clipping at 2.65 prevents exploding gradients during backpropagation.

## 2.5 Objective Function and Evaluation Protocol

The objective function that Optuna optimizes evaluates each hyperparameter configuration through 5-fold cross-validation. Each configuration is evaluated on all 5 folds of the cross-validation split, providing a robust estimate of generalization performance that is not sensitive to any particular train-test split. Within each fold’s training set, a further 80-20 split creates a validation set using participant-level splitting which ensures no participant appears in both train and validation, maintaining the participant-level independence critical for unbiased evaluation.

Training continues for up to 50 epochs per fold but stops early if validation performance does not improve for 12 consecutive epochs. This prevents wasting computation on configurations that have converged or are diverging. Early stopping also acts as implicit regularization—stopping before the model fully fits the training data often improves generalization.

Optuna’s pruning mechanism terminates unpromising trials early, saving computation. After each epoch, the current validation loss is reported to Optuna which compares this intermediate result against other trials at the same training stage. If the current trial is performing significantly worse than the median of previous trials, the trial is terminated. This is particularly valuable because poor hyperparameter configurations often show their inadequacy early in training.

The final score for each fold combines validation loss and generalization gap using the formula: score equals 0.8 times validation loss plus 0.2 times the absolute difference between validation loss

and training loss. This composite objective encourages finding configurations that both achieve low validation error and generalize well with small gap between training and validation performance. The 0.8 to 0.2 weighting prioritizes absolute performance while still considering generalization.

## 2.6 Gaussian Noise Augmentation Search

After identifying optimal network architecture and training hyperparameters, a separate search optimizes the data augmentation strategy—specifically, the level of Gaussian noise to add to input sensors during training. Neural networks can memorize training data rather than learning generalizable patterns, especially with limited training samples. Adding random noise to inputs during training forces the network to learn robust features that work despite measurement uncertainty and sensor variability. Gaussian noise simulates real-world sensor noise, calibration errors, and inter-individual variability, making the model more robust when deployed on new patients or in different environmental conditions. However, too much noise destroys the signal and prevents learning, while too little provides insufficient regularization benefit.

During training (not testing), zero-mean Gaussian noise is added to each IMU sensor channel where the standard deviation is computed per channel as a fraction of that channel’s standard deviation in the training data. The noise fraction is the hyperparameter being optimized. This adaptive scaling ensures that noise levels are appropriate for each channel’s natural variability—accelerometer channels with large dynamic range get proportionally larger noise than channels with small natural variability.

Ten noise levels are tested ranging from 0.0 (no noise) to 0.15 (15 percent of channel standard deviation). For each noise level, two models with different random seeds are trained on each of the 5 folds (10 models total per noise level), using a validation split to assess performance. The validation loss and train-validation gap are tracked. Training is limited to 15 epochs per model to speed up the search.

The results show that noise level 0.1 (10 percent of channel standard deviation) provides the best balance. It achieves the smallest generalization gap while maintaining competitive validation loss. This indicates the model is learning robust, generalizable features rather than memorizing training idiosyncrasies. Higher noise levels continue reducing the gap but start increasing validation loss, suggesting excessive noise begins to corrupt the learning signal.

## 2.7 Final Optimal Hyperparameters

After completing both searches (network architecture and training, plus noise augmentation), the final optimal configuration includes architectural choices of kernels [3, 7] for multi-scale temporal feature extraction, channels [24, 48, 96] for progressive feature expansion through three hierarchical convolutional layers, and 4 attention heads for diverse attention patterns.

Regularization parameters include very low dropout rates of 0.0263 for convolutional layers and 0.0024 for fully connected layers, suggesting other regularization mechanisms provide sufficient regularization. Weight decay is set to 0.000032 providing light L2 penalty. Gaussian noise augmentation at 10 percent of channel standard deviation provides strong regularization without corrupting the signal.

Optimization settings include learning rate of 0.00479, AdamW optimizer combining adaptive learn-

ing rates with proper L2 regularization, batch size of 12 balancing GPU memory and batch normalization stability, and gradient clipping at 2.65 to prevent exploding gradients.

Loss and pooling choices include Huber loss which is robust to outliers and adaptive max pooling for the downsampling strategy.

The very low dropout rates suggest that other regularization mechanisms (weight decay, batch normalization, noise augmentation) provide sufficient regularization, and excessive dropout would hinder learning from limited training data. The small batch size balances GPU memory constraints with providing enough samples for stable batch normalization statistics. The moderate weight decay provides gentle L2 regularization without overly constraining weight magnitudes. The 10 percent Gaussian noise proved to be the optimal level—enough to regularize effectively without corrupting the signal. The architecture with [24, 48, 96] channels and [3, 7] kernels achieves approximately 380,000 parameters, comfortably under the 500,000 limit while providing sufficient model capacity for the task.

This optimal configuration achieves cross-validated performance of approximately 3.0 percent normalized RMSE for joint angle prediction and 3.5 percent for moment prediction, representing state-of-the-art performance for this dataset and demonstrating that systematic hyperparameter optimization can substantially improve model performance compared to hand-tuned configurations.

### 3 Key Insights and Architectural Innovations

#### 3.1 Why Multi-Branch Architecture Outperforms Single-Branch

The multi-branch design with different kernel sizes (3 and 7) provides a critical advantage over single-branch architectures. Gait biomechanics involves events occurring at multiple time scales simultaneously. Impact transients during heel strike happen over 10-20 milliseconds (approximately 1-2 time points in the normalized representation), requiring small receptive fields to capture. Weight transfer patterns during stance phase evolve over 400-600 milliseconds (approximately 40-60 time points), requiring larger receptive fields for context. A single kernel size must compromise between these requirements, either missing fine details with large kernels or lacking contextual information with small kernels.

The parallel branch architecture eliminates this compromise. The kernel-3 branch specializes in rapid transitions and high-frequency components, while the kernel-7 branch captures gradual patterns and provides temporal context. The concatenation and subsequent layers learn to weight these complementary representations appropriately for different prediction tasks. Empirical results show that multi-branch configurations consistently outperform single-branch alternatives by 0.5 to 1.0 percentage points in normalized RMSE, a substantial improvement in biomechanics prediction.

#### 3.2 Attention Mechanism: Learning Phase-Specific Importance

The attention mechanism provides interpretability alongside performance improvements. By examining the learned attention weights, we can understand which gait phases the model considers most important for predictions. Visualizations of attention patterns reveal that for knee flexion angle prediction, the model strongly attends to early stance (loading response) and late stance (terminal stance) phases where knee angle changes most rapidly. For knee moment prediction, attention focuses even more heavily on the loading response phase when body weight transfers to the leg and

moments reach peak values.

This learned importance weighting aligns with biomechanical domain knowledge, providing confidence that the model has learned physiologically meaningful relationships rather than spurious correlations. The attention mechanism essentially implements an adaptive, learned version of phase-specific feature weighting that biomechanics researchers might manually design, but discovered automatically from data. The multi-head structure (4 heads) allows the model to maintain multiple parallel attention patterns, with different heads specializing in different aspects of the temporal sequence.

### 3.3 Downsampling Strategy: Balancing Resolution and Computation

The aggressive downsampling from 101 to 8 time points before attention might seem counterintuitive—why throw away temporal resolution? The key insight is that downsampling occurs after hierarchical feature extraction has already captured important patterns at full temporal resolution. The early convolutional layers operating on 101 time points identify specific gait events and temporal patterns. The attention mechanism then operates on compressed representations of these patterns, learning which events are important rather than resolving fine temporal details.

This design choice reduces attention computational cost by 160-fold while preserving prediction quality. Experiments with less aggressive downsampling (101 to 25 time points) showed minimal performance gains (less than 0.2 percent improvement) while dramatically increasing training time and memory requirements. The 8-point representation appears to be a sweet spot: sufficient to represent major gait phases while enabling efficient attention computation.

The use of strided convolutions rather than pooling for downsampling provides learned, task-specific compression. The convolutional filters learn to emphasize predictive features while suppressing noise during the downsampling process, whereas pooling would apply fixed aggregation rules regardless of feature importance. Ablation studies comparing strided convolutions to max pooling and average pooling showed 0.3 to 0.5 percent performance advantage for strided convolutions.

### 3.4 Hierarchical Channel Expansion: Progressive Abstraction

The channel progression [24, 48, 96] implements progressive abstraction from raw sensor signals to high-level biomechanical features. This exponential expansion (each layer approximately doubles the previous) is a common pattern in successful CNN architectures (ResNet, VGG, etc.) but its effectiveness for biomechanics time-series prediction required empirical validation.

The 24-channel first layer forces the model to learn a compact representation of fundamental patterns. This constraint acts as implicit regularization—the model cannot memorize all training examples with only 24 feature channels, so it must extract generalizable patterns. The 48-channel second layer provides sufficient capacity to combine these fundamental patterns into mid-level representations. The 96-channel third layer offers rich representational capacity for high-level features while remaining within the parameter budget.

Alternative progressions were tested: linear expansion [32, 64, 96] showed slightly worse performance, likely because the compressed first layer (32 channels) provided insufficient capacity for fundamental pattern learning. Larger progressions [32, 64, 128] approached the parameter budget limit and showed signs of overfitting with larger train-validation gaps. The [24, 48, 96] progression

achieved optimal balance between representational capacity and regularization.

### 3.5 Separate Prediction Heads: Axis-Specific Learning

The three separate linear prediction heads for X, Y, and Z axes proved superior to a single shared prediction head in ablation studies. A shared head would force the model to learn a single mapping from pooled features to all three axes simultaneously, potentially conflicting when different axes require different feature combinations. For example, sagittal plane motion (X-axis) during walking is dominated by hip and knee flexion-extension with large ROM and strong coupling to gait cycle phase. Frontal plane motion (Y-axis) involves hip abduction-adduction with smaller ROM but critical importance for balance, potentially coupling more strongly to trunk motion and weight transfer. Transverse plane motion (Z-axis) includes hip and knee rotation with smallest ROM and complex coupling to foot progression angle.

Separate heads allow each axis to learn its optimal feature weighting. The X-axis head might heavily weight features from knee and thigh sensors, while the Y-axis head emphasizes trunk sensor features and ground reaction force patterns. This specialization improved overall prediction accuracy by approximately 0.4 percent compared to a shared prediction head, with particularly large improvements for the typically harder-to-predict Y and Z axes.

### 3.6 Regularization Through Multiple Mechanisms

The optimal configuration employs multiple complementary regularization strategies rather than relying on a single technique. Extremely low dropout (0.026 conv, 0.0024 FC) suggests that dropout’s stochastic regularization is largely redundant with other mechanisms. Batch normalization provides regularization through noise injection (different batch statistics at each iteration) and gradient flow improvement. Weight decay (0.000032) provides continuous L2 penalty discouraging large weights. Gaussian noise augmentation (10 percent) is the strongest regularizer, forcing robustness to input perturbations. Early stopping prevents overfitting by terminating training before complete memorization. Small batch size (12) provides noisy gradient estimates acting as regularization.

This multi-mechanism approach is more effective than strong reliance on any single regularizer. Each mechanism addresses different aspects of overfitting: dropout prevents co-adaptation, weight decay controls parameter magnitudes, noise augmentation enforces input robustness, and early stopping limits optimization time. The hyperparameter search automatically discovered the optimal balance among these mechanisms for the specific dataset characteristics.

## 4 Presentation Strategy and Key Talking Points

### 4.1 Opening: Problem Context

When presenting this work, begin by establishing the clinical motivation. Knee osteoarthritis affects millions of patients and clinical assessment of joint biomechanics currently requires expensive laboratory equipment with optical motion capture and force plates. The goal is to enable clinical-quality biomechanics assessment using only wearable IMU sensors, making it accessible in clinics and home environments. This requires predicting gold-standard optical motion capture joint angles and moments from IMU sensor data, which is challenging because IMU sensors measure segment accelerations and rotations while joint biomechanics depend on complex multi-segment coordination and ground reaction forces.

## 4.2 Architecture Explanation: Bottom-Up Build

Present the architecture as a logical progression solving specific challenges:

Start with the input: 42 IMU channels over 101 time points representing one gait cycle.

Explain the multi-branch design: different gait events occur at different time scales, so we need multiple kernel sizes operating in parallel. Branch 1 with kernel-3 captures rapid events, Branch 2 with kernel-7 captures gradual patterns.

Describe hierarchical learning: three convolutional layers with progressive channel expansion (24, 48, 96) learn increasingly abstract features from raw sensor signals to biomechanical patterns.

Justify the aggressive downsampling: processing 101 time points through attention would be computationally prohibitive. We downsample to 8 points using learned strided convolutions, reducing computation by 160-fold while preserving essential information because feature extraction has already occurred.

Explain attention in intuitive terms: different phases of gait have different importance for predictions. Heel strike matters much more than mid-swing for predicting peak knee moment. Attention learns these importance weights automatically, telling the model where to focus.

Describe the prediction heads: three separate linear layers for X, Y, Z axes because each anatomical plane has different characteristics and optimal feature combinations.

## 4.3 Attention Mechanism: Visual Explanation

The attention mechanism often requires the most careful explanation because it is conceptually sophisticated. Use this analogy: imagine you are predicting tomorrow's weather. You have temperature, humidity, and pressure measurements from the past week. Attention is like asking which past days are most relevant for tomorrow's prediction—yesterday's measurements probably matter most, while measurements from 7 days ago matter less. The model learns these relevance weights automatically.

For gait biomechanics, attention learns that heel strike is highly relevant for predicting knee moment, while mid-swing is less relevant. The multi-head mechanism (4 heads) is like having multiple experts each looking for different patterns: one expert tracks impact events, another tracks weight transfer, another tracks limb positioning. Their insights are combined for the final prediction.

## 4.4 Hyperparameter Optimization: The Search Process

Explain hyperparameter optimization as systematic automated search replacing tedious manual trial-and-error. The search space is enormous: even 10 hyperparameters with 5 options each gives 10 million configurations. Optuna uses smart search strategies (Bayesian optimization with TPE) that build a model of which hyperparameters work well, then suggest promising configurations to try next. This is much more efficient than random search.

Emphasize the two-stage search: first optimize architecture and training (kernels, channels, learning rate, etc.), then separately optimize data augmentation (noise level). This separation makes the search more manageable and interpretable.



Highlight key findings: very low dropout rates indicate other regularization mechanisms (batch norm, noise, weight decay) are sufficient. The 10 percent Gaussian noise level provides optimal regularization. The [24, 48, 96] channel progression balances capacity and generalization.

## 4.5 Results: Performance and Validation

Present results emphasizing cross-validated performance metrics. The 5-fold cross-validation with participant-level splitting ensures robust generalization assessment—no participant appears in both train and test sets. Achieving approximately 3.0 percent normalized RMSE for angles and 3.5 percent for moments represents clinically useful accuracy, approaching inter-rater reliability of human evaluators in some studies.

Compare to baselines if available: simpler architectures (single-branch CNNs), traditional machine learning (Random Forests, SVMs), or previous published work on similar tasks. The multi-branch attention architecture should show clear improvements, validating the architectural innovations.

Discuss practical implications: 3 percent error on joint angles means predictions within 2-3 degrees for typical walking movements, which is sufficient for many clinical applications like tracking disease progression or evaluating interventions. For moments, 3.5 percent error translates to small absolute differences in Nm that are acceptable for most clinical decisions.

## 4.6 Limitations and Future Work

Acknowledge limitations honestly: the dataset contains only 44 participants from a single laboratory, limiting generalization to different populations and collection protocols. The model predicts only knee joint biomechanics, while comprehensive gait analysis requires hip and ankle as well. Predictions are made in time-normalized space (gait cycle percentage) rather than real-time, requiring gait event detection for deployment.

Propose future directions: expand dataset to multiple sites and populations to improve generalization, extend to multi-joint prediction (hip, knee, ankle simultaneously), develop real-time prediction capability with online gait event detection, investigate transfer learning from large public datasets, and explore model interpretability through attention visualization and feature importance analysis.

## 4.7 Closing: Impact and Significance

Conclude by returning to the clinical impact. This work demonstrates that wearable sensors combined with deep learning can achieve clinical-quality joint biomechanics predictions, enabling assessment outside expensive motion capture laboratories. This technology could transform management of knee osteoarthritis by enabling frequent, convenient biomechanics monitoring during rehabilitation, objective measurement of treatment outcomes, and personalized intervention based on individual movement patterns. The systematic hyperparameter optimization methodology is generalizable to other biomechanics prediction tasks, providing a template for future research.

## 5 Technical Deep Dives for Q&A

### 5.1 Why 101 Time Points Specifically

The 101 time points (0 through 100 percent in 1 percent increments) is a biomechanics convention for time-normalized gait data. This representation allows comparison across different walking speeds and stride times by expressing time as percentage of gait cycle completion. The choice of 101 rather than 100 comes from including both endpoints: 0 percent (initial contact) and 100 percent (subsequent initial contact), which are technically the same instant but helps with certain visualization and processing conventions. Alternative representations like 51 points (2 percent increments) or 201 points (0.5 percent increments) are possible but 101 provides good balance between temporal resolution and computational cost.

### 5.2 Why Not RNN/LSTM for Temporal Sequences

Recurrent architectures like LSTMs and GRUs are designed for sequential processing where each time step depends on hidden states from previous time steps. For gait biomechanics, the time-normalized representation means we have access to the entire sequence simultaneously, not requiring sequential processing. CNNs with appropriate receptive fields can capture temporal dependencies through convolution operations which are more computationally efficient and easier to parallelize on GPUs. Additionally, RNNs can suffer from vanishing gradients over long sequences, whereas CNNs with skip connections and proper initialization train more stably. Empirical comparisons in biomechanics literature generally show CNNs matching or exceeding RNN performance for time-normalized gait prediction tasks.

### 5.3 Attention Versus Simple Pooling

Why not just use weighted average pooling with learned weights instead of the full attention mechanism? The key difference is that attention weights are input-dependent (dynamically computed based on the specific input sequence) while learned pooling weights are input-independent (fixed after training). Attention can adapt to different gait patterns, emphasizing heel strike for one patient while emphasizing push-off for another based on their individual movement characteristics. Fixed pooling weights cannot provide this adaptability. The performance difference is empirically significant: attention-based models show 0.5 to 0.8 percent improvement over fixed weighted pooling in ablation studies.

### 5.4 Handling Variable Number of Sensors

What if some sensor locations are unavailable (for example, only 5 sensors instead of 7)? The architecture as designed requires all 42 input channels and would need retraining if sensor configuration changes. Two approaches address this: train separate models for each sensor configuration (practical if only a few configurations are common), or design a sensor-agnostic architecture using set-based encoders or mask tokens for missing sensors. The latter approach trades some performance for flexibility and requires careful architectural redesign with attention mechanisms that can handle variable-length sensor sets.

### 5.5 Transfer Learning Potential

Could this model be pre-trained on a large dataset then fine-tuned for specific populations? Yes, transfer learning is promising for biomechanics. A model pre-trained on healthy adults could be

fine-tuned with small datasets of pathological gait patterns. The early convolutional layers learn general features (sensor signal patterns, basic gait events) that transfer well across populations, while later layers and prediction heads learn population-specific patterns requiring fine-tuning. This approach could dramatically reduce data requirements for new applications. However, careful validation is needed because negative transfer (pre-training hurting rather than helping) can occur if source and target populations differ substantially in movement patterns.